Advanced Databases

Homework 2: Deadlocks, Restarts & Livelocks

Aamir Zulfiquar Ansari, Maximilian Moderegger, Thomas Martin Pfister

Setup, Task distribution, contribution & time management

- IntelliJ 2026.2.3 / VSCode 1.144
- Java v27

Setup, Task distribution, contribution & time management

- IntelliJ 2026.2.3 / VSCode 1.144
- Java v27
- PostgreSQL 18
- JDBC 42.7.14

Setup, Task distribution, contribution & time management

- IntelliJ 2026.2.3 / VSCode 1.144
- Java v27
- PostgreSQL 18
- JDBC 42.7.14

- “Winged” the Task
- Met up on Discord
- Contribution was equal

Challenges Faced

- Finding suitable criterium for Live-locks
- Achieving Live-locks
- Logging into Github (2FA troubles)

Something new

- LOCK TABLE... IN EXCLUSIVE MODE
  - Initial thought was to track and handle starving transactions ourselves.
  - Tablelock does that -> Running transactions finish, new ones queue behind.
- Max learned about the existence of Google-slides

Blocking Demo

Concept RestartBlocking

- Wrap task execution in a while-loop

```
while(!committed) {//restart the transaction until it commits
                try {
                    con.setAutoCommit(false);
                    con.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
                    int q = 0;//always restart with the first query, the rollback undid
                                all of them
                    ...
                    con.commit();
                    committed = true;
                    int num_finished = tasks_finished.incrementAndGet();
                    System.out.println("Done task "+task+" after "+attempt+" attempt(s).
I am finisher number                                    "+num_finished);
                } catch (SQLException e) {
                    if (DEADLOCK_CODE.equals(e.getSQLState())) {// we were chosen as
deadlock victim -> roll back and
            try again
                        tasks_restarted.incrementAndGet();
                        attempt++;
                        System.out.println("Task "+task+" deadlock -> restart");
                        try {
                            con.rollback();
                        } catch (SQLException ex) {
                            throw new RuntimeException(ex);
                        }
                    } else {//any other error is a real problem -> give up
                        System.err.println("Task "+task+" "+e.getMessage());
                        break;
                    }
                }
            }
```

Concept RestartBlocking

- Wrap task execution in a while-loop
- On SQLException:

  check if error code is ‘40P01’ (DEADLOCK DETECTED)

```
while(!committed) {//restart the transaction until it commits
                try {
                    con.setAutoCommit(false);
                    con.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
                    int q = 0;//always restart with the first query, the rollback undid
                                all of them
                    ...
                    con.commit();
                    committed = true;
                    int num_finished = tasks_finished.incrementAndGet();
                    System.out.println("Done task "+task+" after "+attempt+" attempt(s).
I am finisher number                                    "+num_finished);
                } catch (SQLException e) {
                    if (DEADLOCK_CODE.equals(e.getSQLState())) {// we were chosen as
deadlock victim -> roll back and
            try again
                        tasks_restarted.incrementAndGet();
                        attempt++;
                        System.out.println("Task "+task+" deadlock -> restart");
                        try {
                            con.rollback();
                        } catch (SQLException ex) {
                            throw new RuntimeException(ex);
                        }
                    } else {//any other error is a real problem -> give up
                        System.err.println("Task "+task+" "+e.getMessage());
                        break;
                    }
                }
            }
```

Concept RestartBlocking

- Wrap task execution in a while-loop
- On SQLException:

  check if error code is ‘40P01’ (DEADLOCK DETECTED)

- If deadlock:
  - Increment Restart Counter
  - Rollback Transaction

```
while(!committed) {//restart the transaction until it commits
                try {
                    con.setAutoCommit(false);
                    con.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
                    int q = 0;//always restart with the first query, the rollback undid
                                all of them
                    ...
                    con.commit();
                    committed = true;
                    int num_finished = tasks_finished.incrementAndGet();
                    System.out.println("Done task "+task+" after "+attempt+" attempt(s).
I am finisher number                                    "+num_finished);
                } catch (SQLException e) {
                    if (DEADLOCK_CODE.equals(e.getSQLState())) {// we were chosen as
deadlock victim -> roll back and
            try again
                        tasks_restarted.incrementAndGet();
                        attempt++;
                        System.out.println("Task "+task+" deadlock -> restart");
                        try {
                            con.rollback();
                        } catch (SQLException ex) {
                            throw new RuntimeException(ex);
                        }
                    } else {//any other error is a real problem -> give up
                        System.err.println("Task "+task+" "+e.getMessage());
                        break;
                    }
                }
            }
```

RestartBlocking Demo

Concept NoLiveLocks

- Check if starvation criterium has been exceeded for a task e.g. 3 attempts

```
while(!committed) {//restart the transaction until it commits
    try {
        ...
        if (attempt > MAX_ATTEMPTS_BEFORE_LOCK) {//starving task: wait for running
                transactions, block new ones, then run alone
            System.out.println("Task "+task+" runs alone (attempt
                    "+attempt+")");
            executeSQL("LOCK TABLE blocking_data IN EXCLUSIVE MODE;", con);
        }
        ...
        con.commit();
        committed = true;
        int num_finished = tasks_finished.incrementAndGet();
        System.out.println("Done task "+task+" after "+attempt+" attempt(s).
    I am finisher number "+num_finished);
    } catch (SQLException e) {
        if (DEADLOCK_CODE.equals(e.getSQLState())) {//we were chosen as
                    deadlock victim -> roll back and try again
            tasks_restarted.incrementAndGet();
            attempt++;
            System.out.println("Task "+task+" deadlock -> restart");
            try {
                con.rollback();
                Thread.sleep(rand.nextInt(MAX_BACKOFF_MS));//random pause, so the
                                                    victims don't restart in lockstep
            } catch (SQLException | InterruptedException ex) {
                throw new RuntimeException(ex);
            }
        } else {//any other error is a real problem -> give up
            System.err.println("Task "+task+" "+e.getMessage());
            break;
        }
    }
}
```

Concept NoLiveLocks

- Check if starvation criterium has been exceeded for a task e.g. 3 attempts
- On starvation detected: Get table lock until the task is finished

```
while(!committed) {//restart the transaction until it commits
    try {
        ...
        if (attempt > MAX_ATTEMPTS_BEFORE_LOCK) {//starving task: wait for running
                transactions, block new ones, then run alone
            System.out.println("Task "+task+" runs alone (attempt
                    "+attempt+")");
            executeSQL("LOCK TABLE blocking_data IN EXCLUSIVE MODE;", con);
        }
        ...
        con.commit();
        committed = true;
        int num_finished = tasks_finished.incrementAndGet();
        System.out.println("Done task "+task+" after "+attempt+" attempt(s).
    I am finisher number "+num_finished);
    } catch (SQLException e) {
        if (DEADLOCK_CODE.equals(e.getSQLState())) {//we were chosen as
                    deadlock victim -> roll back and try again
            tasks_restarted.incrementAndGet();
            attempt++;
            System.out.println("Task "+task+" deadlock -> restart");
            try {
                con.rollback();
                Thread.sleep(rand.nextInt(MAX_BACKOFF_MS));//random pause, so the
                                                    victims don't restart in lockstep
            } catch (SQLException | InterruptedException ex) {
                throw new RuntimeException(ex);
            }
        } else {//any other error is a real problem -> give up
            System.err.println("Task "+task+" "+e.getMessage());
            break;
        }
    }
}
```

Concept NoLiveLocks

- Check if starvation criterium has been exceeded for a task e.g. 3 attempts
- On starvation detected: Get table lock until the task is finished
- Add Jitter when restarting other tasks

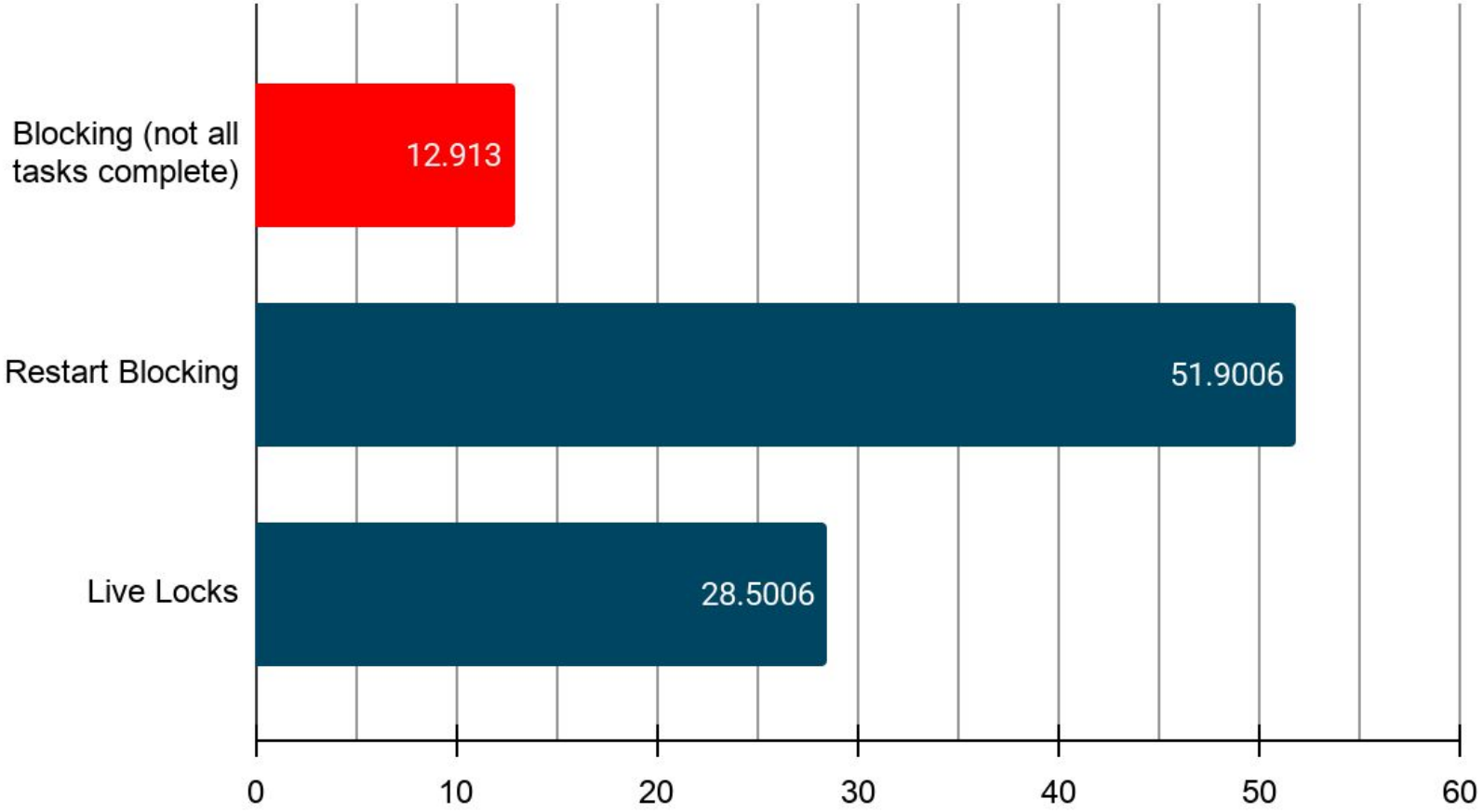
```
while(!committed) {//restart the transaction until it commits
    try {
        ...
        if (attempt > MAX_ATTEMPTS_BEFORE_LOCK) {//starving task: wait for running
                transactions, block new ones, then run alone
            System.out.println("Task "+task+" runs alone (attempt
                    "+attempt+")");
            executeSQL("LOCK TABLE blocking_data IN EXCLUSIVE MODE;", con);
        }
        ...
        con.commit();
        committed = true;
        int num_finished = tasks_finished.incrementAndGet();
        System.out.println("Done task "+task+" after "+attempt+" attempt(s).
    I am finisher number "+num_finished);
    } catch (SQLException e) {
        if (DEADLOCK_CODE.equals(e.getSQLState())) {//we were chosen as
                    deadlock victim -> roll back and try again
            tasks_restarted.incrementAndGet();
            attempt++;
            System.out.println("Task "+task+" deadlock -> restart");
            try {
                con.rollback();
                Thread.sleep(rand.nextInt(MAX_BACKOFF_MS));//random pause, so the
                                                    victims don't restart in lockstep
            } catch (SQLException | InterruptedException ex) {
                throw new RuntimeException(ex);
            }
        } else {//any other error is a real problem -> give up
            System.err.println("Task "+task+" "+e.getMessage());
            break;
        }
    }
}
```

NoLiveLocks Demo

Benchmark configuration

- Average results of 3 runs each
- Number of Tasks: 30
- Number of Queries per Task: 10
- Number of Threads: 4

Average runtime in seconds



Any Questions?